

AA-08

Identity, Permissions, PII and Security Charter

Four actor classes, eighty-eight permissions, and the last-four-only disclosure regime

30 August 2026 · Rev 1.0 — baseline

ACTOR CLASSES	PERMISSION STRINGS	STEP-UP PERMISSIONS
4	88	22
SYSTEM ROLES	MASKED PII FIELDS	PLAINTEXT DISCLOSURE PATHS
17	14	1

This document is a charter, not a description. Every rule below is numbered so it can be cited in a code review, and every rule carries the mechanism that enforces it and the consequence of breaking it. On identity, authorisation and personal data this document outranks every other document in the set: where AA-04 or AA-05 appears to grant something this charter withholds, this charter is right and the other document has a defect. On their own subjects — contract shapes, screen behaviour, schema — they outrank this one. Read AA-00 sections 0.5 and 0.9 for the invariants and the risk register that this document implements.

8.1 WHAT THIS CHARTER BINDS

Four actor classes, three portals, one API, and the precedence rule

Everything in the programme reduces to 4 classes of actor talking to one API through 5 namespaces. There is no second backend, no admin-only database user, and no service that holds a credential a portal could also hold. That is invariant I-01, and it is the reason this charter can be short: there is exactly one place where identity is established, exactly one place where a permission is evaluated, and exactly one place where a field is masked. A rule that lives in one place can be enforced; a rule repeated in four places is a wish.

The 88 permission strings and 17 roles in sections 8.5 and 8.6 are the normative copy. AA-04 prints a permission beside every route and AA-05, AA-06 and AA-07 print one beside every screen, and both are convenience restatements. If a permission string appears in those documents and not in section 8.5, the route does not ship: the CI permission sweep in AA-01 fails the build because the string is not a member of the Permission union.

The precedence rule is worth stating flatly because it will be tested during week five when somebody wants a reveal button. On identity, authentication, authorisation and personal data, this document wins and the other document has a defect to be raised as an issue. On its own subject — the shape of a response body, the behaviour of a screen, the columns of a table — the other document wins. No document in the set may relax a rule numbered here.

8.1.1 The four actor classes

ACTOR	ENTITY	AUTH	AUD	SURFACE	TOTP	NOTES
customer	User	Mobile OTP (WhatsApp, SMS fallback)	atv_app	Mobile app	No	Farmers and other buyers. Existing. Unchanged.
staff	StaffUser	Email + Argon2id password + mandatory TOTP	atv_admin	Admin portal	Yes, always	Atavishaala employees: ops, catalog, support, finance, agronomists, admins.
partner	PartnerUser	Email + Argon2id password, TOTP for finance	atv_partner	Supplier portal	Conditional	Employees of a verified product company (partner_orgs).
fleet	FleetUser	Email or mobile + Argon2id, TOTP for finance	atv_fleet	Fleet portal	Conditional	Employees and riders of a verified delivery company (fleet_orgs).

The asymmetry in the TOTP column is the first honest compromise in the document and it is defended in 8.3.2. Staff are employees on managed browsers and can be told to install an authenticator; a supplier packing clerk on a shared warehouse terminal cannot, and a rider on a bike certainly cannot. Conditional means mandatory for anyone who can reach a bank account or a payout view, and off for everyone else — the condition is the role, not the preference.

Two of these four classes are outside the organisation and can self-serve their way to a login. A product company and a delivery company are counterparties, not colleagues, and one of them will eventually be hostile or merely careless; 8.12 enumerates what a hostile registration reaches before a human approves it.

RULE 8.1.A ONE IDENTITY PER SURFACE

A person who is both a staff member and a partner owner holds two unrelated accounts with two unrelated credentials. There is no account linking and no actor switching. Enforcement: the aud claim is checked by requireActor before any handler runs (I-02), so a linked identity would have nothing to switch into. Consequence of breaking it: a single phished credential reaches two namespaces, and the audit trail can no longer say which hat the person wore.

RULE 8.1.B THE CHARTER OUTRANKS THE IMPLEMENTATION

If the code allows something section 8.4 or 8.7 forbids, the code is the bug, regardless of how long it has been in production. Enforcement: the PII sweep task P9-BE1-01 and the authorisation matrix test P9-BE1-02 in week nine read this document as the expected value. Consequence of breaking it: the sweep becomes a description of current behaviour rather than a test, which is how PII regimes rot.

8.2 AUTHENTICATION AND THE TOKEN MODEL

Three portal login flows, four audiences, and where the two tokens live

Authentication answers one question — which row in which identity table is this — and then stops. It does not answer what the caller may do; that is section 8.4, evaluated separately, after the audience check, on every

request. Keeping the two apart is why a fleet token presented to the admin namespace produces 401 ACTOR_MISMATCH rather than a permission-denied row: the request is rejected before a permission is ever looked up, so a scanning attacker learns nothing about the admin permission space from a stolen fleet token. AA-04 section 4.2 carries the request and response shapes; what follows is the rule set behind them.

8.2.1 Staff login: password plus TOTP, no exceptions

1. POST /admin/auth/login takes email and password. Argon2id verify against staff_users.password_hash, m=19456 KiB, t=2, p=1, tuned for the 512 MiB Cloud Run instance. A correct password does not return a session. It returns a challenge token with a 300-second life and nothing else. A wrong password returns 401 after a constant-time delay with no hint about which half failed, and increments staff_users.failed_count.
2. POST /admin/auth/totp with the challenge token and six digits is the only code path that mints a staff session. staff_users.totp_enrolled_at must be NOT NULL before the session is issued, so a staff row without a confirmed factor cannot reach step two at all. This applies to all 9 admin roles including SUPER_ADMIN.
3. The response returns the access token in the body and sets the refresh token as a cookie. GET /admin/me then returns the staff row, the flattened effective permission array of section 8.4, and the timestamp of the last verified factor.

The obvious hole in a mandatory-TOTP rule is the first account, and the usual answer is a bypass flag that nobody ever removes. We do not have one. Migration M1 seeds a single SUPER_ADMIN row from BOOTSTRAP_ADMIN_EMAIL with status INVITED, no password hash and no TOTP secret. That row cannot authenticate: there is nothing to verify against. The first login is therefore not a login but an invite acceptance — the same single-use token flow a partner owner uses — and enrolment is a required step inside it, so the account transitions straight from INVITED to ACTIVE with a confirmed factor and has never existed in a password-only state. BOOTSTRAP_ADMIN_EMAIL is then removed from the environment; it is consumed exactly once.

The rejected alternative was an environment flag such as ALLOW_TOTP_BYPASS=true for the first boot. It lost because a flag that grants password-only access to a SUPER_ADMIN seat is a standing backdoor whose only protection is that somebody remembered to flip it back, and because it would have to be readable by the running service, which means it is one Cloud Run revision away from being true again in production.

8.2.2 Partner and fleet login, and the narrower rider session

A partner user signs in with email, password and the org slug. The slug in the request body is not a violation of the tenancy rule in 8.4.C, because no token exists yet and therefore no claim to contradict; it disambiguates one person who works for two supplier orgs under the same email address, is matched against partner_orgs once, and is never consulted again. Every subsequent request is scoped by the partner_org_id inside the minted token. There is no self-registration on this namespace at all: a partner org exists because a staff member invited it.

A fleet office user signs in the same way. A FLEET_RIDER does not, because a rider has a phone and no email address they read. POST /fleet/auth/otp/request sends six digits over the WhatsApp-first channel the customer app already uses, with SMS fallback; the code is stored as a bcrypt hash in otp_challenges.code_hash and appears nowhere else — not in the row, not in a log line, not in a Sentry breadcrumb. That route is idempotency-keyed so a rider double-tapping in weak signal does not burn two codes. Five failures lock the mobile for 15 minutes, which is deliberately looser than the delivery-OTP lockout because a rider locked out at 07:00 cannot start work and will call ops, and ops cannot help.

The rider session is the narrowest in the system: the token carries the rider seat so GET /fleet/jobs returns only that rider_id's jobs, the role holds 5 permissions and none of them reads money, riders or org settings. A stolen rider phone leaks a handful of masked job cards plus the addresses of that rider's currently open jobs, the residual risk recorded in AA-07 section 7.2.

8.2.3 The token model

PROPERTY	ACCESS TOKEN	REFRESH TOKEN
Audience	One of atv_app atv_admin atv_partner atv_fleet, in the aud claim, checked by requireActor	Bound to the same audience and to a family id; useless on any other namespace
Algorithm	RS256, signed with JWT_PRIVATE_KEY, verified against the JWKS at /.well-known/jwks.json	Opaque random token; only its hash is stored server-side
Lifetime	900 seconds for the three portals, 600 for the mobile app (ACCESS_TTL_SECONDS)	7 days admin, 14 partner and fleet, 30 app (REFRESH_TTL_DAYS)

PROPERTY	ACCESS TOKEN	REFRESH TOKEN
Refresh window	None. It expires and the portal silently refreshes	Rotating: presenting it returns a new one and invalidates the presented one immediately
Rotation	Re-minted on every refresh, so a revoked permission takes at most 900 seconds to bite	One-time use. A second presentation of a used token is treated as theft
Family revoke	Not revocable individually; it is short-lived instead	Replay of a used token revokes the whole family and forces a full re-login
Storage	JavaScript memory in the portal tab. Never localStorage, never sessionStorage, never a readable cookie	Secure, HttpOnly, SameSite=Strict, host-only cookie scoped to the API origin, named per namespace

8.2.4 Rule 8.2.A — where the two tokens live, and what we rejected

The refresh token lives in a Secure, HttpOnly, SameSite=Strict, host-only cookie scoped to the API origin. The access token lives in a JavaScript variable inside the running tab and is lost on reload, which costs one silent refresh call. Neither token is ever written to localStorage or sessionStorage, and no portal reads document.cookie. This is invariant I-10 and it is enforced mechanically: the aa/no-web-storage ESLint rule errors on any reference to those three APIs in a portal repo, and a CI grep over the three repos fails the build independently of the linter.

The rejected alternative was the common one — refresh token in localStorage, read by the client and sent as a bearer header. It is easier: no cookie configuration, no CORS credentials, no CSRF question, and it survives a reload for free. It loses to cross-site scripting. Any script that executes in the portal origin — a compromised npm dependency, an operator pasting something into the console, a stored-XSS field we failed to escape — can read localStorage synchronously and exfiltrate a token with a week of life on it in a single fetch. The same script against an HttpOnly cookie can only make requests from the live tab while it is open. We accepted the CSRF question in exchange, and section 8.11.4 answers it. Moving the access token into the cookie as well was also rejected, for a duller reason: the mobile app is not a browser and has no cookie jar we control.

8.3 TOTP, ENROLMENT AND THE FRESH-FACTOR GATE

Invariant I-09, and an audit of which endpoints deserve the gate

I-09 STAFF AUTHENTICATION REQUIRES TOTP; FINANCE ACTIONS RE-PROMPT

A StaffUser cannot complete login without a registered TOTP factor. Refund approval, payout release and permission changes require a fresh TOTP within the last 5 minutes. Enforcement: staff_users.totp_enrolled_at NOT NULL is enforced before the session is issued. requireFreshFactor(300) guards the twenty-four mutating step-up routes. Four GET routes share a step-up permission and are not re-challenged, because the gate is applied on the mutation and not on the read.

8.3.1 Enrolment, recovery codes and logout

Enrolment happens inside invite acceptance, before the account is usable, and not at first login. The distinction matters: an account that can log in once without a factor is an account an attacker can log in to once without a factor, and the window is however long the new hire takes to read their email. The secret is generated server-side, rendered once as a QR code with TOTP_ISSUER set to "Atavishaala Admin", and stored in staff_totp.secret_cipher under the same AES-256-GCM envelope as every other encrypted column. It is never returned again by any endpoint, so a lost authenticator is a reset, not a re-display.

Ten single-use recovery codes are shown once, on the same screen, with an explicit instruction to write them down. Only bcrypt hashes are kept, in staff_totp.recovery_hashes. There is no second chance to view them and no endpoint that regenerates them without a fresh factor, which means the failure mode of losing both the phone and the codes is a manual reset by the other SUPER_ADMIN. That is the reason the SUPER_ADMIN seat has a two-holder minimum in practice even though staff.manage enforces a two-holder maximum: one holder is an availability risk, three is a blast-radius risk.

staff_totp.last_used_step records the 30-second step of the last accepted code, and a code from a step already used is rejected even if it is otherwise valid; without that, an attacker who observes six digits in a phishing proxy has the rest of the window to reuse them. Ten consecutive failures set staff_users.locked_until. Ten rather than five, because clock skew on a desktop authenticator produces a run of failures that is not an attack, and a

five-attempt logout on a mandatory factor turns a skewed clock into an outage.

8.3.2 Rule 8.3.A — conditional TOTP for counterparties, and why

Partner and fleet TOTP is mandatory for any user whose role holds a permission in the finance or org groups — PARTNER_OWNER, PARTNER_FINANCE, FLEET_OWNER, FLEET_FINANCE — and off for everyone else. It is conditional on the role, never on the user's preference, and the condition is evaluated at role assignment: promoting a PARTNER_CATALOG user to PARTNER_FINANCE forces enrolment before the new role takes effect.

Mandatory TOTP for every counterparty user was tried and rejected. A warehouse packing seat is shared by whoever is on shift, and the observed outcome of forcing an authenticator onto a shared terminal was a single shared secret taped to the monitor — a factor that is neither second nor a factor. The compromise concentrates the requirement where the loss is monetary and irreversible, and accepts that a stolen packing-seat password can accept and pack shipments. That is containable: those actions are all audited, reversible by ops, and cannot move money.

8.3.3 The fresh-factor gate: 29 endpoints, counted from the contract

requireFreshFactor(300) does not ask whether the caller has a factor. It asks when they last proved it, reading the stamp GET /admin/me exposes. If the last verified factor is older than 300 seconds the request never reaches the handler: the response is 401 STEP_UP_REQUIRED with a challenge object, the portal opens its step-up dialog, and the original request is replayed with the same body and the same Idempotency-Key. Because the first attempt never created an idempotency row, the replay is a first attempt rather than a replay.

The gated set is not a list somebody maintains. It is derived: 22 of the 88 permission strings carry the step-up flag (17 admin, 2 partner, 3 fleet), and every route that declares one of them is gated by construction. Filtering the 164 portal routes of AA-04 against those 22 strings yields 29 endpoints, listed below. Three other numbers exist in the fact base and none of them is 29: I-09 says twenty-eight, the note column of AA-04 marks 24, and task card P2-BE1-03 says eleven. The derivation is the truth, because the guard is attached to the permission and not to the prose; the other three are documentation that drifted as the enumeration grew, and all three need a patch in aa_facts before the week-two freeze. The rule itself has not changed.

NS	METHOD	PATH	DECLARED PERMISSION	MARKED IN AA-04	TEST LIMB
admin	PATCH	/admin/variants/:id/price	catalog.price.override	yes	money
admin	POST	/admin/shipments/:id/transition	order.transition.force	yes	money
admin	POST	/admin/jobs/:id/cancel	job.cancel	yes	NONE - see 8.3.5
admin	POST	/admin/refunds/:id/approve	refund.approve	yes	money
admin	POST	/admin/wallet:userId/credit	wallet.credit	yes	money
admin	POST	/admin/wallet:userId/debit	wallet.debit	yes	money
admin	POST	/admin/payout-runs/:id/release	payout.run.release	yes	money
admin	POST	/admin/cod/:id/reconcile	cod.reconcile	yes	money
admin	PUT	/admin/partners/:id/rate-card	commission.write	yes	money
admin	PATCH	/admin/partners/:id	partner.status.write	no (defect)	authority
admin	POST	/admin/partners/:id/status	partner.status.write	yes	authority
admin	POST	/admin/partner-bank/:id/verify	partner.bank.verify	yes	money
admin	POST	/admin/fleets/:id/status	fleet.status.write	yes	authority
admin	POST	/admin/advisory/:id/sla	advisory.sla.override	yes	NONE - see 8.3.5
admin	POST	/admin/customers/:id/status	customer.status.write	yes	authority
admin	POST	/admin/staff	staff.manage	yes	authority
admin	PUT	/admin/staff/:id/roles	staff.manage	yes	authority
admin	PUT	/admin/roles/:id/permissions	role.write	yes	authority
admin	GET	/admin/settings	settings.write	no (defect)	authority

NS	METHOD	PATH	DECLARED PERMISSION	MARKED IN AA-04	TEST LIMB
admin	PUT	/admin/settings/:key	settings.write	yes	authority
partner	PUT	/partner/bank	p.bank.manage	yes	money
partner	GET	/partner/users	p.user.manage	no (defect)	authority
partner	POST	/partner/users	p.user.manage	yes	authority
partner	POST	/partner/users/:id/disable	p.user.manage	yes	authority
fleet	GET	/fleet/cod	f.cod.declare	no (defect)	money
fleet	POST	/fleet/cod/:id/declare	f.cod.declare	yes	money
fleet	PUT	/fleet/bank	f.bank.manage	yes	money
fleet	GET	/fleet/users	f.user.manage	no (defect)	authority
fleet	POST	/fleet/users	f.user.manage	yes	authority

8.3.4 Rule 8.3.B — the three-part test for what earns a step-up

An endpoint is step-up gated if and only if it does at least one of three things: it moves money, it changes who can do things, or it discloses personal data. Everything else is gated by a permission and audited, and that is enough. The test exists because the alternative — gating by how sensitive an action feels — produces a gate on every second button, and a gate that fires thirty times a day is answered by an authenticator app left open beside the keyboard, which is strictly worse than no gate because it looks like a control.

Two things follow from the wording. First, irreversibility alone does not earn a gate: rejecting a KYC document is irreversible in feeling but recoverable in one click, so it is not gated, while verifying a bank account is gated because the next thing that happens to that account is a payout release. Second, 300 seconds is a window and not a per-action prompt. A partner manager suspending four suppliers in one sitting types six digits once. Per-action re-auth was rejected for the reason above; 300 seconds is still short enough that an unlocked laptop in a shared office is not a standing grant.

8.3.5 Where the gated set and the test disagree

Auditing the 29 gated endpoints against the test finds failures in both directions, and they are recorded here rather than quietly reconciled.

PERMISSION OR ROUTE	DIRECTION	FINDING AND DISPOSITION
job.cancel	gated, fails test	Cancelling a delivery job moves no money, changes no authority and discloses nothing. It is gated because an ops manager cancelling a job strands a shipment a customer has paid for. Keep the gate; the test is the floor, not the ceiling.
advisory.sla.override	gated, fails test	Waiving an advisory SLA clock fails all three limbs. This one is a genuine over-gate: it exists because the permission sat beside the finance overrides when the matrix was written. Disposition: keep for the pilot, review at the first agronomist complaint.
GET /admin/settings, GET /partner/users, GET /fleet/cod, GET /fleet/users	gated by accident	4 read routes declare a write permission (settings.write, p.user.manage, f.cod.declare, f.user.manage) and are therefore gated as a side effect. An operator will meet a TOTP prompt on a list page, which reads as a bug and will be reported as one. Fix in AA-04, not here: split the read permission out of the declaration. Corrected count 25.
order.cancel	ungated, passes test	Cancelling a paid order reverses a payment or voids a COD collection, so it moves money. It is not gated. This is the most defensible omission to close before go-live.
payout.line.withhold	ungated, passes test	Withholding a line withholds money from a supplier, without the release that is gated. Mitigated only by the append-only ledger and the audit row. Close it.
partner.user.manage, rider.suspend	ungated, passes test	Both change who can do things, inside a counterparty org. p.user.manage is gated inside the partner namespace while the admin equivalent is not, which is an inconsistency rather than a decision.
f.job.manifest	ungated, passes test	The one endpoint in the programme that discloses plaintext PII is not step-up gated. A rider on a bike cannot produce an authenticator code one-handed and a gate people route around is a fiction. Accepted deviation, not an oversight: the controls are the register, the job scope and the 240-minute window of section 8.8.

8.4 THE PERMISSION MODEL

String shape, DENY-wins, union across roles, and the flattened effective array

A permission is a dotted lower-case string in one of three shapes: resource.action, resource.sub.action, or a namespace-prefixed form for the counterparty portals where every string begins p. or f. so that a partner permission and an admin permission can never be confused by eye or by a grep. The prefix is what lets the CI permission sweep assert that no route under /api/v1/partner declares a string from the admin set. There is no wildcard and no implied hierarchy, so holding catalog.write does not imply catalog.read.

The type is a closed union of 88 string literals in packages/shared, and that is the load-bearing decision of this section. A permission that is not in the union does not compile, so a route cannot declare one that does not exist, a seed cannot grant one, and — the point of 8.7.3 — a permission we deliberately never name cannot be granted by mistake. The rejected alternative, a free-text permission column with runtime validation, catches the same mistakes four hours later in a staging log instead of in the editor.

8.4.1 Rule 8.4.A — DENY wins, unconditionally

role_permissions carries an effect column with two values, ALLOW and DENY. A staff user may hold several roles through staff_role_grants, and resolveEffective() takes the union of the ALLOW entries and then subtracts every DENY entry found in any held role. DENY wins over ALLOW in a different role, over ALLOW in the same role, and over SUPER_ADMIN. There is no precedence order between roles to reason about and no most-specific-match rule, because both of those are features whose failure mode is a security decision nobody can predict by reading the grants.

What DENY buys that removing a role does not is the ability to compose an exception without inventing a role. A finance manager who must not approve refunds while an investigation is open keeps FINANCE and receives a one-entry role carrying DENY refund.approve. Removing FINANCE instead would take away every other Money and Payouts permission the person needs, and cloning FINANCE minus one permission into FINANCE_RESTRICTED creates a role that will drift from its parent the next time a permission is added, silently, because nothing links them.

The cost is real: an effective permission set is now the result of a computation rather than a lookup, and "why can this person not do that" needs a tool. GET /admin/me returns the resolved array, and the staff detail screen in AA-05 prints the resolution with the DENY source named, because a DENY nobody can find is indistinguishable from a bug.

8.4.2 Rule 8.4.B — the portal holds permissions, never role names

GET /admin/me returns a flat array of resolved permission strings. It does not return role names, and no portal contains the substring SUPER_ADMIN in a conditional. Every gate in the three portals is can('refund.approve'), a membership test against that array, because a role name in a client conditional is a permission model with two implementations shipping on different schedules: add a permission to FINANCE in a migration and the server allows it while the portal still hides the button.

The array is advisory, not authoritative. It decides what is rendered; it never decides what is allowed. Every request is re-evaluated server-side by requirePermission() regardless of what the client believed, so a hand-edited array in a debugger changes what the operator sees and nothing about what the API will do. A permission removed mid-session takes at most 900 seconds to bite, because the array is re-minted on each access-token refresh.

8.4.3 Rule 8.4.C — tenant scope comes only from the token

partner_org_id and fleet_org_id are read from the verified token claim and from nowhere else. A request that carries either in its body, query or path is rejected with 400 SCOPE_IN_REQUEST before the handler runs — rejected, not ignored, because a silently ignored tenant parameter is a client that believes it is scoping and is not, and that belief will eventually be load-bearing in a report. The only exception is the org slug at login, where no token exists yet (8.2.2).

The mechanism is two-layered. aa/no-tenant-in-input errors on any Zod schema under a partner or fleet route that declares either field, so the request shape cannot accept one; and the middleware rejects it anyway, so a schema written by hand outside the linted path still fails. Admin routes are exempt because a staff member legitimately addresses any org, and that is precisely why every admin mutation is audited with the target named (8.13).

8.5 THE PERMISSION MATRIX: 88 STRINGS

56 admin, 17 partner, 15 fleet. This is the normative copy

These 88 rows are the Permission union. The STEP-UP column is the source of the gated set derived in 8.3.3, so a change to a value in that column changes what requireFreshFactor(300) guards, without anyone editing a list of routes. A dash means the endpoint is protected by the permission and the audit row alone.

Read the groups as ownership boundaries rather than as menus. The recurring pattern is that creating a thing and releasing it are separate permissions — refund.initiate and refund.approve, payout.run.create and payout.run.release, catalog.write and catalog.moderate — so that a role can be composed which prepares work it cannot commit. The separation is real at the permission level and only advisory at the human level: nothing stops one person holding both halves, and with a team of four somebody will. Section 8.15 lists that as a gap.

8.5.1 admin / Catalog — 6 permissions

catalog.write and catalog.moderate are separate because the person who fixes a supplier's title must not be the person who approves that supplier onto the shelf; CATALOG_MANAGER is the only role holding moderate. catalog.price.override is the one step-up here because a price is what a customer pays, and media.delete is split out because deleting a photo cannot be undone.

PERMISSION	WHAT IT ALLOWS	STEP-UP
catalog.read	View products, variants, categories, brands	-
catalog.write	Create and edit house products and taxonomy	-
catalog.moderate	Approve, reject or delist a partner-submitted product	-
catalog.price.override	Override an MRP or selling price on a live variant	yes
catalog.stock.adjust	Post a manual stock correction with a reason code	-
catalog.media.delete	Remove a product image or video from Cloudinary	-

8.5.2 admin / Orders — 6 permissions

order.read is the widest read in the system and is held by four of the nine admin roles, which is why the masking rule of 8.7 has to be unconditional rather than role-dependent. order.transition.force is the escape hatch for a shipment stuck in the FSM; it writes a shipment state and never orders.status, which is derived by reduceOrderStatus() and is not writable by any permission in this table.

PERMISSION	WHAT IT ALLOWS	STEP-UP
order.read	View orders, shipments and the event timeline	-
order.cancel	Cancel an order or a single shipment before handover	-
order.shipment.reassign	Move items between shipments or re-split an order	-
order.transition.force	Force a shipment transition the normal FSM forbids	yes
order.note.write	Add an internal note visible to staff only	-
order.return.review	Approve or reject a return with photo evidence	-

8.5.3 admin / Delivery — 5 permissions

job.assign and job.rebroadcast are separate acts: assigning hands a job to a named fleet, rebroadcasting re-enters it into the offer waves and can cost the original fleet a job it thought it held. job.pod.review is the dispute seat and is the only permission in this group that reads a geo point, rounded to three decimals on the way out.

PERMISSION	WHAT IT ALLOWS	STEP-UP
job.read	View delivery jobs, offers, riders and POD records	-
job.assign	Manually assign an unassigned job to a fleet	-
job.rebroadcast	Push a job back into the offer auction at a chosen wave	-
job.cancel	Cancel an accepted job and return the shipment to the pool	yes
job.pod.review	Inspect POD geo and photo evidence, flag a suspect delivery	-

8.5.4 admin / Money — 6 permissions

Three of these six are step-up gated and all three move money out. `refund.initiate` creates a `PENDING_APPROVAL` row and is not gated, which is the correct reading of the test in 8.3.4: nothing has moved yet. There is no wallet cash-out permission because there is no cash-out — holding customer balances redeemable for cash is a prepaid payment instrument and needs an RBI licence we do not have, so `wallet.credit` and `wallet.debit` adjust a balance spendable only inside the platform.

PERMISSION	WHAT IT ALLOWS	STEP-UP
payment.read	View payments, gateway references and settlement status	-
refund.initiate	Raise a refund request against a payment or shipment	-
refund.approve	Approve a refund and release it to gateway or wallet	yes
wallet.credit	Post a goodwill or compensation credit to a customer wallet	yes
wallet.debit	Post a corrective debit against a customer wallet	yes
coupon.write	Create, edit or expire a coupon	-

8.5.5 admin / Payouts — 6 permissions

`payout.run.create` assembles a `DRAFT` run and `payout.run.release` sends it; only the second is gated. `commission.write` is gated because a rate card is effective-dated and one edit changes every future settlement for that supplier, silently, with no per-order trace other than the pricing snapshot. `payout.line.withhold` is the omission argued in 8.3.5: it withholds money without the gate that releasing it carries.

PERMISSION	WHAT IT ALLOWS	STEP-UP
payout.read	View accrued payout lines and past runs for both kinds	-
payout.run.create	Assemble a draft weekly payout run	-
payout.run.release	Lock and release a payout run for bank transfer	yes
payout.line.withhold	Withhold a single line from a run with a reason	-
cod.reconcile	Match declared COD against received COD and close a period	yes
commission.write	Publish a new effective-dated commission rate card	yes

8.5.6 admin / Partners — 6 permissions

`partner.kyc.review` is not gated and `partner.bank.verify` is, and that pair is the clearest illustration of the test: rejecting a document is one click to undo, while verifying an account is the last check before money leaves. `partner.status.write` is gated because suspension delists every SKU the supplier holds, and restoring the supplier does not relist them.

PERMISSION	WHAT IT ALLOWS	STEP-UP
partner.read	View product companies, their users and their documents	-
partner.invite	Invite a new product company and its first owner user	-
partner.kyc.review	Verify or reject a submitted KYC document	-
partner.status.write	Activate, suspend or terminate a product company	yes
partner.user.manage	Reset, disable or re-role a partner user	-
partner.bank.verify	Mark a partner bank account as verified	yes

8.5.7 admin / Fleets — 7 permissions

The largest group, because a delivery company is the counterparty we trust least and therefore control most. `fleet.tier.write` is the quiet one: a tier decides which wave a fleet is offered jobs in, so it allocates revenue between fleets and arguably passes the money limb of the test while carrying no gate. `fleet.area.approve` exists although cut C-8 removed the approval workflow, so for the pilot an admin edits areas directly under it. `rider.suspend` is the only admin permission acting on an individual person, which is why AA-07 requires a reason string.

PERMISSION	WHAT IT ALLOWS	STEP-UP
<code>fleet.read</code>	View delivery companies, riders, vehicles and service areas	-
<code>fleet.invite</code>	Invite a new delivery company and its first owner user	-
<code>fleet.kyc.review</code>	Verify or reject fleet KYC and insurance documents	-
<code>fleet.status.write</code>	Activate, suspend or terminate a delivery company	yes
<code>fleet.area.approve</code>	Approve a service-area or rate-card change request	-
<code>fleet.tier.write</code>	Change a fleet tier, which changes its broadcast priority	-
<code>rider.suspend</code>	Suspend a rider across all future offers	-

8.5.8 admin / Advisory — 5 permissions

`advisory.claim` and `advisory.reassign` are queue mechanics rather than authority: an agronomist claims a case to stop two experts answering the same farmer. AGRONOMIST holds every string here except `sla.override`, and holds `customer.read` so the caller can be identified — which under 8.7 means a first name and the last four digits of a mobile, and nothing more. `sla.override` is the over-gate named in 8.3.5.

PERMISSION	WHAT IT ALLOWS	STEP-UP
<code>advisory.read</code>	View advisory requests, media and notes	-
<code>advisory.claim</code>	Claim a request from the queue as the handling agronomist	-
<code>advisory.note.write</code>	Record the callback outcome and recommendation	-
<code>advisory.reassign</code>	Move a claimed request to another agronomist	-
<code>advisory.sla.override</code>	Extend or waive the SLA clock with a written reason	yes

8.5.9 admin / Customers — 3 permissions

The smallest group and the one most likely to be misread. `customer.address.read` does not grant an address: it grants locality, city and pincode, which is what an ops agent needs to judge whether a delivery failure is plausible. No string in this table returns a full address or a phone number, and no combination of them does either, because the masking sits at the ORM boundary below all of them (8.7.4).

PERMISSION	WHAT IT ALLOWS	STEP-UP
<code>customer.read</code>	View customer profiles, masked contact and order history	-
<code>customer.status.write</code>	Suspend or restore a customer account	yes
<code>customer.address.read</code>	View a customer address, masked to locality and pincode	-

8.5.10 admin / Platform — 6 permissions

`staff.manage` carries the check that keeps SUPER_ADMIN to two holders; it is the only permission in the set that enforces a cardinality rule. `role.write` exists in the union even though cut C-4 removed the role editor screen, because the seed in M1 writes `role_permissions` through the same code path and because DENY grants are applied with it by SQL during the pilot. `audit.read` is read-only by construction: there is no `audit.write` string, and 8.13 explains why the table cannot be written by the API at all. `export.create` produces masked CSV capped at 5 000 rows.

PERMISSION	WHAT IT ALLOWS	STEP-UP
staff.read	View staff users and their effective permissions	-
staff.manage	Invite, re-role, suspend or offboard a staff user	yes
role.write	Create a custom role or change a role permission set	yes
settings.write	Change a platform setting or flip a feature flag	yes
audit.read	Read the audit log and the PII disclosure register	-
export.create	Queue a masked CSV export of a list surface	-

8.5.11 partner namespace — 17 permissions in 5 groups

Every string here begins p. and is inert on any other namespace: a partner token is checked by requireActor before a permission is evaluated, so presenting one to /api/v1/admin fails as ACTOR_MISMATCH rather than as a permission denial. The set is small because a supplier does exactly four things — lists, packs, receives returns, and reconciles — and every one is scoped to the partner_org_id in the token claim under rule 8.4.C, so there is no cross-tenant permission to get wrong; the tenancy is structural.

GROUP	PERMISSION	WHAT IT ALLOWS	STEP-UP
partner / Catalog	p.catalog.read	View own products and their moderation state	-
	p.catalog.draft	Create or edit a draft product and submit it for review	-
	p.catalog.stock	Update stock quantity on an approved variant	-
	p.catalog.price	Propose a price change, which re-enters moderation	-
partner / Fulfilment	p.shipment.read	View shipments assigned to this organisation	-
	p.shipment.accept	Accept a shipment and commit to a ready-by time	-
	p.shipment.pack	Move a shipment through PACKING to READY_FOR_PICKUP	-
	p.shipment.shortship	Declare a short-ship with a reason, triggering a refund	-
	p.manifest.print	Print the pick list and the shipment label	-
partner / Returns	p.return.read	View returns and RTO coming back to this organisation	-
	p.return.receive	Confirm physical receipt of a returned or RTO shipment	-
	p.return.dispute	Dispute a return within 48 hours of receipt	-
partner / Finance	p.payout.read	View accrued lines, past runs and the running ledger	-
	p.invoice.download	Download the self-billed commission invoice for a run	-
	p.bank.manage	Add or replace the settlement bank account	yes
partner / Org	p.user.manage	Invite, re-role or disable a user in this organisation	yes
	p.kyc.submit	Upload or replace a KYC document	-

p.catalog.price is separate from p.catalog.draft because a price change on a live SKU takes effect on the shelf while a draft waits for catalog.moderate, so they carry different blast radii despite editing the same row. p.shipment.shortship is its own string rather than a flag on pack because short-shipping writes a pricing adjustment, and an operator who may pack must not necessarily be able to change what the customer is charged. p.bank.manage and p.user.manage are the two step-up strings, matching the finance-and-authority rule of 8.3.4.

8.5.12 fleet namespace — 15 permissions in 5 groups

The fleet set is the one written with an untrusted holder in mind, and it shows in what is absent. There is no f.customer.read, no f.order.read and no f.payment.read: a delivery company is told where to take a parcel and what to collect, and nothing about who bought what for how much. f.job.manifest is the single string in all three namespaces whose response contains plaintext personal data, and section 8.8 is entirely about the controls wrapped around it.

GROUP	PERMISSION	WHAT IT ALLOWS	STEP-UP
fleet / Offers	f.offer.read	See live offers addressed to this fleet	-

GROUP	PERMISSION	WHAT IT ALLOWS	STEP-UP
fleet / Jobs	f.offer.respond	Accept or decline an offer before it expires	-
	f.job.read	View this fleet's jobs and their masked drop details	-
	f.job.assign	Attach a rider and a vehicle to an accepted job	-
	f.job.manifest	Open the job manifest, which discloses the drop address	-
	f.job.scan	Record pickup and handover scans	-
	f.pod.write	Capture proof of delivery: OTP, geo point and optional photo	-
fleet / Fleet	f.job.fail	Record a failed attempt with an outcome code	-
	f.rider.manage	Add, edit, deactivate a rider; upload rider documents	-
	f.vehicle.manage	Add or retire a vehicle and update insurance expiry	-
fleet / Finance	f.area.request	Request a service-area or rate-card change	-
	f.payout.read	View earnings lines, runs and the fleet ledger	-
	f.cod.declare	Declare cash collected and record the remittance reference	yes
	f.bank.manage	Add or replace the payout bank account	yes
fleet / Org	f.user.manage	Invite, re-role or disable a user in this fleet	yes

f.offer.respond is not step-up gated even though accepting an offer commits the fleet to a job with a penalty attached, because an offer lives ten minutes and a TOTP prompt inside that window loses the race — a worse outcome than the risk. f.cod.declare is gated on both its routes, which is how the accidental gate on GET /fleet/cod in 8.3.5 arose. The FLEET_RIDER role is composed from 5 of these 15 strings and holds nothing from Finance, Fleet or Org.

8.6 THE 17 SYSTEM ROLES

9 admin, 4 partner, 4 fleet, all seeded, none editable in the pilot

A role is a named bundle of permission entries and nothing else. It carries no implicit capability, no hierarchy and no inheritance: FLEET_MANAGER is not a superset of anything, it is a list. All three sets are seeded by migration M1 with is_system true, and the summaries below are the seed. A staff user may hold several roles and the result is resolved by rule 8.4.A; a partner or fleet user holds exactly one role_code, because a four-role set with a single signatory seat has nothing to compose.

8.6.1 admin roles (9)

ROLE CODE	PERMISSIONS	PURPOSE AND BOUNDARY
SUPER_ADMIN	Everything except raw PII, which no role grants	Founder and technical lead only. Two holders maximum, enforced by a check in staff.manage.
OPS_MANAGER	All Orders, Delivery, Advisory read; job.assign, job.rebroadcast, order.cancel, order.return.review	Day-to-day exception handling. The busiest role.
OPS_AGENT	order.read, order.note.write, job.read, customer.read, advisory.read	Read-mostly support seat. Cannot cancel, refund or reassign.
CATALOG_MANAGER	All Catalog including moderate and price.override; partner.read	Owns the shelf. The only role that approves partner listings.
FINANCE	All Money and Payouts; partner.bank.verify, fleet read	Refund approval and the weekly release. Cannot touch catalog or orders.
PARTNER_MANAGER	All Partners; catalog.read, payout.read	Supplier relationship owner: onboarding, KYC, commission, suspension.
FLEET_MANAGER	All Fleets and Delivery; payout.read	Delivery relationship owner: onboarding, tiers, service areas, rider suspension.
AGRONOMIST	All Advisory except sla.override; customer.read	The expert console seat. Sees crop media and the masked caller ID, nothing else.
AUDITOR	Every *.read permission and audit.read. No write permission at all.	For a reviewer or an external accountant. Deliberately cannot export raw anything.

Two rows deserve reading twice. SUPER_ADMIN says "everything except raw PII, which no role grants" — that is not a courtesy, it is the type-level fact of 8.7.3, and the two-holder maximum is enforced inside staff.manage rather than by convention. AUDITOR holds every read and no write, which makes it the correct seat for an external accountant and the wrong seat for anyone who needs to fix something; it can also enumerate masked records at volume, which is gap G-07.

8.6.2 partner roles (4) and fleet roles (4)

ROLE CODE	PERMISSIONS	PURPOSE AND BOUNDARY
PARTNER_OWNER	All 17 partner permissions	The signatory. Exactly one per org, transferable by admin.
PARTNER_CATALOG	p.catalog.*, p.shipment.read	Lists and prices products. No money, no users.
PARTNER_OPS	p.shipment.*, p.return.*, p.manifest.print, p.catalog.stock	Warehouse seat. Accepts, packs, hands over, receives returns.
PARTNER_FINANCE	p.payout.read, p.invoice.download, p.bank.manage, p.shipment.read	Reconciles settlement. Cannot change the shelf or move a shipment.
FLEET_OWNER	All 15 fleet permissions	The signatory. Exactly one per org.
FLEET_DISPATCH	f.offer.*, f.job.read, f.job.assign, f.rider.manage, f.vehicle.manage, f.area.request	The control-room seat: accepts offers and allocates riders.
FLEET_FINANCE	f.payout.read, f.cod.declare, f.bank.manage, f.job.read	Declares COD and reconciles earnings.
FLEET_RIDER	f.job.read, f.job.manifest, f.job.scan, f.pod.write, f.job.fail	Mobile-only seat. Sees only jobs assigned to that rider, and only inside the window.

CUT C-4 — NO ROLE EDITOR, AND WHAT THAT COSTS

Ship the nine system roles only; no role editor screen. Saved: 4 dev-days. Rationale: Nine roles cover every seat we can name today. The consequence is not that exceptions are impossible but that they are made in SQL rather than in a screen: a one-entry DENY role, written by hand against role_permissions, with no audit_log row because the change did not arrive through the admin router. staff_roles still carries is_system and the table is shaped for custom roles, so this is a missing screen and not a missing model — but until the screen exists, the authorisation state of the system can be changed without a trace, and that is worse than the four dev-days saved. Mitigation for the pilot: every such grant is written as a migration file, so the git history is the audit trail.

8.7 THE PII REGIME

Three rules, invariants I-06 and I-07, and the column triple that implements them

I-06 ADMIN SURFACES SHOW THE LAST FOUR CHARACTERS ONLY

No phone, GSTIN, PAN, Aadhaar, bank account, IFSC, vehicle registration or driving licence is rendered beyond its last four characters in the admin portal. There is no reveal affordance and no role, including SUPER_ADMIN, that unmask. Enforcement: There is deliberately no pii.read.raw permission in the Permission type union. A Prisma client extension masks unconditionally at the ORM boundary. Five ESLint rules and a pre-commit grep for ten consecutive digits.

I-07 DELIVERY ADDRESS RELEASE IS JOB-SCOPED, WINDOW-SCOPED AND LOGGED

A fleet principal sees a customer address only via GET /fleet/jobs/:id/manifest, only while the job is ACCEPTED..OUT_FOR_DELIVERY, only inside the delivery window, and every read appends a pii_disclosures row. Enforcement: The manifest handler is the single code path that calls unmaskAddress(); it is wrapped by discloseOrThrow() which writes the register row inside the same transaction.

8.7.1 Rule 8.7.A — the last four characters, everywhere, for everyone

No phone number, GSTIN, PAN, Aadhaar, bank account number, IFSC, vehicle registration, driving licence number, payment reference or payout UTR is rendered beyond its last four characters in any of the three portals, for any role, on any screen, in any export, in any log line and in any error message. Not "except for support". Not "except for finance". The rule is stated as an absolute because every partial version of it that we drafted turned into a list of exceptions that grew, and because a masked field with one exception has to be reasoned about at every call site while a masked field with none does not.

8.7.2 Rule 8.7.B — there is no reveal affordance

There is no eye icon, no long-press, no double-click, no "show full number" link, no tooltip and no copy-to-clipboard that copies more than what is on screen. This is a separate rule from 8.7.A because the two fail differently: 8.7.A can be broken by a careless template, while 8.7.B is broken by a deliberate feature that somebody asks for with a good reason in week five. Writing it down as its own numbered rule means the answer is a charter revision rather than a ticket.

8.7.3 Rule 8.7.C — no permission grants plaintext, because none is named

There is no `pii.read.raw` string in the Permission union, and there is no other string that returns an unmasked field. That absence is the enforcement. The union is a closed type of 88 literals (8.4), so a route cannot declare a permission that does not exist and a seed cannot grant one; a reviewer asking "which role can unmask a phone number" gets a compile error rather than a policy answer. A permission that cannot be named cannot be granted by mistake, cannot be added to a role by a well-meaning migration, and cannot be requested by a screen.

The rejected alternative was the ordinary one: define `pii.read.raw`, grant it to nobody, and rely on the grant table staying empty. It loses because an empty grant is a runtime fact maintained by discipline, and the first genuine escalation at 23:00 will be resolved by adding one row to `staff_role_grants` — which is a two-second fix that no test fails on and no reviewer sees. The absent type is the only version of this rule that survives its first emergency.

8.7.4 The 14 governed fields, and what each actor class sees

FIELD	WHERE IT LIVES	STORAGE	STAFF SEES	PARTNER SEES	FLEET SEES
Customer mobile	<code>users.phone</code>	cipher + last4 + HMAC index	last 4 only	never	never
Customer name	<code>user_profiles.full_name</code>	plaintext (not a masked field)	full	first name only	first name only, on the manifest
Delivery address	<code>addresses.*</code>	line1/line2 cipher; locality, city, pincode plaintext	locality + city + pincode	locality + pincode	full, job-scoped and logged
Delivery coordinates	<code>addresses.geo_lat/lng</code>	cipher, 3-decimal rounded on read	never	never	full, job-scoped and logged
Delivery OTP	<code>delivery_jobs.otp_hash</code>	bcrypt hash only, digits never stored	never	never	never (entered, not read)
Login OTP	<code>otp_challenges.code_hash</code>	bcrypt hash only	never	never	never
Payment reference	<code>payments.gateway_ref</code>	cipher + last4	last 4 only	never	never
Partner GSTIN / PAN	<code>partner_orgs.*</code>	cipher + last4 + HMAC index	last 4 only	own, last 4 only	never
Bank account / IFSC	<code>partner_bank_accounts, fleet_bank_accounts</code>	cipher + last4	last 4 only	own, last 4 only	own, last 4 only
Payout UTR	<code>payout_runs.utr_reference</code>	cipher + last4	last 4 only	last 4 only	last 4 only
Rider licence number	<code>riders.dl_*</code>	cipher + last4 + HMAC index	last 4 only	never	own riders, last 4 only
Vehicle registration	<code>vehicles.reg_*</code>	cipher + last4 + HMAC index	last 4 only	never	own, last 4 only
KYC document image	Cloudinary private folder	signed URL, 5-minute expiry	view only, no download, logged	own uploads	own uploads
Staff / partner / fleet email	<code>staff_users, partner_users, fleet_users</code>	plaintext	full (business contact, not customer PII)	own org	own org

Two rows are not masked and both are deliberate. A customer's full name is plaintext because a name is what makes a support call possible and masking it produces a screen nobody can work from; the counterparties see a first name only, which is enough to hand a parcel over. Business email addresses are plaintext because a partner contact is a commercial counterparty, not a consumer. Two rows are not readable by anyone: both OTP hashes. A delivery OTP is entered and compared, never displayed, so no screen and no endpoint returns it, and the digits are never stored — only a bcrypt hash — which means an operator genuinely cannot read a code out to a rider over the phone, and RB-03 exists because of that.

8.7.5 The column triple, and where masking is enforced

```
users.phone_cipher    bytea    AES-256-GCM(PII_ENC_KEY), iv + tag prefixed. Never selected by a
                    portal-facing query; no read path returns it.
users.phone_last4     text      The rendered value. Denormalised on write, so masking costs no
                    decryption on any list screen.
users.phone_index     text      HMAC-SHA256(PII_INDEX_HMAC_KEY, normalised value). Equality search
                    only: hash the needle, compare. No range, no LIKE, no sort.

prisma.$extends({ result: { $allModels: { ...mask every *_cipher to null, expose *_last4 } } })
```

The masking extension is attached to the single Prisma client the API constructs, unconditionally, with no flag, no bypass parameter and no second unmasked client. A handler that wants a cipher column cannot get one by asking; the field arrives as null. That is a stronger position than a mask-on-render helper, because a helper is something a new endpoint can forget and an extension is something a new endpoint cannot opt into. The one escape hatch in the entire codebase is `unmaskAddress()`, called from exactly one file, the subject of section 8.8.

The searchable index is why an ops agent can find an order by a customer's full phone number without any screen ever showing it: the number is typed into the search box, hashed in the handler, and compared. The cost is stated in 8.10 — `PII_INDEX_HMAC_KEY` is never rotated, because rotating it invalidates every index column at once and there is no re-index job in the plan.

THE KEY-MANAGEMENT GAP, STATED PLAINLY

`PII_ENC_KEY` is a 32-byte AES-256-GCM key held as a base64 string in a Cloud Run environment variable. There is no KMS, no envelope encryption, no per-record data key and no hardware boundary, because Cloud KMS costs money and this programme runs at Rs 0 per month recurring (I-14). What this buys is real: a leaked Neon backup or a stolen read-replica credential yields ciphertext. What it does not buy: anyone who can read the running service configuration can decrypt everything, and that includes anyone with the GCP project role that lists revisions. Rotation requires a re-encrypt job that does not exist yet. This is gap G-01 and it is the most consequential compromise in the document.

8.8 THE SINGLE DISCLOSURE EXCEPTION

`GET /fleet/jobs/:id/manifest`, and the register that makes it survivable

One endpoint in the programme returns plaintext personal data. `GET /fleet/jobs/:id/manifest` releases the delivery address (line1, line2, locality, city, pincode), the geo coordinates rounded to three decimals, and the receiver's first name, to a fleet principal, for one job, while that job is between `ACCEPTED` and `OUT_FOR_DELIVERY`, and for at most 240 minutes (`MANIFEST_WINDOW_MINUTES`) after it goes out for delivery. Outside any one of those three conditions the response is 403 and no address is computed, not merely not rendered. It exists because a parcel cannot be delivered to a masked address; every other read of an address in the system returns locality, city and pincode.

```
GET /fleet/jobs/9f21/manifest      Authorization: Bearer <fleet token>

requireActor(atv_fleet) -> f.job.read + f.job.manifest -> job.fleet_org_id == claim ?
job.status in (ACCEPTED, PICKED_UP, IN_TRANSIT, OUT_FOR_DELIVERY) ?
now() - job.out_for_delivery_at <= 240 min  (or not yet out for delivery) ?

discloseOrThrow({ actor, subject_user_id, field: "address", reason: "DELIVERY_MANIFEST",
                  ref_type: "delivery_job", ref_id }) {
  tx.pii_disclosures.create(...)    <-- same transaction
  unmaskAddress(address_id)         <-- the only caller of the only escape hatch
}                                   rollback of either rolls back both
```

The register write and the decryption are in one transaction, in that order, so a disclosure without an audit row is not a policy violation but a physical impossibility: if the insert fails the read is rolled back, and if the read fails the row is rolled back with it. That is the whole design. A log line written after the fact can be lost, batched, dropped under load or skipped by an early return; a row in the same transaction cannot.

REASON CODE	WHEN IT IS WRITTEN	ACTOR	WHAT IS RELEASED
DELIVERY_MANIFEST	A fleet principal opened the manifest for an accepted job.	fleet	address, coordinates, receiver name
KYC_REVIEW	A staff reviewer opened a KYC document image.	staff	document image

REASON CODE	WHEN IT IS WRITTEN	ACTOR	WHAT IS RELEASED
SUPPORT_ESCALATION	A step-up-authorized ops action needed the address to resolve a dispute.	staff	address
LEGAL_REQUEST	A documented law-enforcement or regulatory request was served.	staff	any field, named in the row
EXPORT_JOB	A masked CSV export was generated. Recorded even though it is masked.	staff	nothing raw; recorded for completeness

RULE 8.8.A THE CUSTOMER PHONE NUMBER IS NEVER RELEASED, UNDER ANY CODE PATH

No endpoint in any namespace returns a customer mobile number in plaintext. Not the manifest, not a support screen, not an export, not an error payload, not a webhook. There is no reason code for it, no permission for it, and no function that decrypts users.phone_cipher outside the OTP send path, which passes the number to the WhatsApp or SMS provider and never returns it upward. The consequence is accepted and it is not small: risk R-04 is a rider who cannot reach a customer, and the workaround is a WhatsApp template send triggered from the job screen plus RB-03. The named paid fix is an Exotel or Kaleyra masked-call bridge at roughly Rs 0.60 per minute plus Rs 1 500 per month, which needs a written waiver against I-14.

The compensating controls are three and they are all detective rather than preventive, which is the honest way to describe them. GET /admin/pii-disclosures is a register an admin reads: every row names the actor, the subject, the field, the reason and the job, it is never purged by any job and it is retained 24 months. A daily check flags any fleet whose disclosure count per accepted job rises above its own trailing average, because the meaningful signal is not volume but the ratio of reads to jobs the fleet was actually given. And the manifest is job-scoped, so nothing enumerates: a fleet cannot ask for a hundred addresses, only a hundred times for the hundred jobs it holds.

A fleet exfiltrating addresses at scale has a specific shape in the data, worth writing down so the person reading the register knows what to look for: accept rate near 100 % across every wave including jobs outside its usual pincodes, a manifest read on nearly every accepted job within seconds of acceptance rather than at pickup, a rising rate of jobs failed or cancelled after the manifest was opened, and disclosure rows clustered in bursts. Any two together is a conversation; three is grounds to set the fleet to offer-ineligible under 8.14.

8.9 DEFENCE IN DEPTH IN THE CODE

8 lint rules, two pre-commit greps, one ORM extension, one log redactor

RULE	WHAT IT CATCHES	CLASS OF BUG IT PREVENTS
aa/no-raw-pii-render	A JSX expression that renders a field name matching /phone gstin pan aadhaar account ifsc licence dl _reg utr/ without passing through mask() or last4().	A template that renders a value the API should never have sent, and would have shipped an unmasked field the moment masking regressed.
aa/no-tenant-in-input	A Zod schema under a partner or fleet route that declares partner_org_id or fleet_org_id.	Cross-tenant read or write from a request parameter — the classic multi-tenant hole.
aa/no-status-write	An object literal containing a status key inside prisma*.update or .updateMany, outside */fsm.ts.	A derived field written directly, so orders.status stops agreeing with its shipments and no query can tell which is right.
aa/no-web-storage	Any reference to localStorage, sessionStorage or document.cookie in a portal repo.	A refresh token readable by any script in the origin (I-10).
aa/no-direct-http	An import of axios, ky, or a bare fetch call in a portal repo. All calls go through the kit client.	A call that skips the 401 interceptor, the single-flight refresh and the Idempotency-Key, giving three portals three auth behaviours.
aa/require-idempotency	A route registered with meta.money or meta.offer that lacks the idempotency middleware.	A double-charge or a double-accept from a retried request.
aa/no-float-money	An arithmetic operator applied to an identifier ending in _paise outside packages/shared/money.	Rounding drift in a ledger that has no UPDATE path to correct it.
aa/no-prisma-in-portal	An import from @prisma/client or a DATABASE_URL reference in a portal repo.	A database credential in a client bundle (I-01).

Three layers sit underneath those rules. A pre-commit hook runs the same eslint invocation and a grep for ten consecutive digits inside a template literal or JSX text, which catches the hard-coded test phone number that a lint rule keyed on field names cannot see; the same grep runs in CI, because a hook is a local courtesy and a contributor can pass --no-verify. The Prisma masking extension is attached to the client the API constructs, so it

is unconditional and there is no unmasked client to reach for. And Pino is configured with redaction paths for every governed field name plus authorization, cookie, set-cookie, password, code and otp, so a request-body log of a login attempt records the shape and not the secret.

The layering argument is the point, and it is not that the layers are individually strong. Each one is defeatable: a lint rule is silenced with a disable comment, a hook is skipped, a grep is evaded by string concatenation, the ORM extension is bypassed by `$queryRaw`, and the redactor misses a field it was never told about. What the stack buys is that no single mistake gets through, and that every route around it is a deliberate act which shows up as a diff a reviewer can see — an `eslint-disable` line naming the rule, a `--no-verify` in a commit, a `$queryRaw` in a handler. That turns "somebody was careless" into "somebody chose to", which is the only meaningful line a codebase can draw.

8.10 SECRETS: 15 OF THEM, AND WHICH HAVE NO ROTATION PROCEDURE

Where each one lives, and why VITE_ prefixed values are public by definition

SECRET	STORE	ROTATION PROCEDURE
DATABASE_URL	Cloud Run env var only	Reset the Neon role password, update the variable, deploy a revision. The pooled endpoint makes this near-zero downtime. Rehearsed once.
DIRECT_URL	GitHub Actions secret; Cloud Run env var	Same reset as above. Also held as a GitHub secret because prisma migrate runs in CI, so both copies must change together.
JWT_PRIVATE_KEY	Cloud Run env var only	Publish the new public key in the JWKS beside the old, mint with the new, drop the old after the longest refresh TTL has elapsed. Not rehearsed. This is the riskiest rotation in the list.
JWT_PUBLIC_KEY	Cloud Run env var only	Served from <code>/.well-known/jwks.json</code> ; rotates with the private key and never alone.
PII_ENC_KEY	Cloud Run env var only	Requires a re-encrypt job over every <code>*_cipher</code> column in the database. That job does not exist. Gap G-01.
PII_INDEX_HMAC_KEY	Cloud Run env var only	None, deliberately. Rotating it invalidates every <code>*_index</code> column simultaneously and no re-index job exists, so a rotation would silently break phone and GSTIN search. Gap G-02.
CLOUDINARY_API_KEY	Provider console; Cloud Run env var	New pair issued in the console; key and secret swap together. Already-issued signed URLs stay valid for their 5-minute life.
CLOUDINARY_API_SECRET	Provider console; Cloud Run env var	As above. Never present in any portal build; see below.
RAZORPAY_KEY_SECRET	Provider console; Cloud Run env var	Console rotation. RAZORPAY_KEY_ID is public and unchanged.
RAZORPAY_WEBHOOK_SECRET	Provider console; Cloud Run env var	Console rotation, but the verifier must accept old and new for one deploy or in-flight webhooks fail closed and retry into the dead-letter path.
WHATSAPP_TOKEN	Provider console; Cloud Run env var	Meta permanent token. Record the expiry date in the Monday sync notes and rotate in week 5 rather than discovering it on a failed OTP.
SMS_2FACTOR_KEY	Provider console; Cloud Run env var	Console rotation. Fallback path only, so a bad rotation stays invisible until WhatsApp fails over — test it deliberately.
BREVO_API_KEY	Provider console; Cloud Run env var	Console rotation. Blast radius is invites and password resets only.
FCM_SERVICE_ACCOUNT	Provider console; Cloud Run env var	Issue a second service-account key, swap the variable, then delete the first in IAM. The only rotation here that is safe by construction.
CRON_SHARED_SECRET	GitHub Actions secret; Cloud Run env var	Change both sides in one commit. A mismatch fails <code>/internal/cron/*</code> closed, which the missed-run alert catches within one interval.

Rule 8.10.A: every VITE_ prefixed variable is public by definition. Vite inlines those values into the bundle at build time, so a VITE_ variable is not configuration that happens to be readable — it is source code shipped to a browser, and treating it as anything else is the mistake. The 6 portal variables are therefore all things that appear in devtools without embarrassment: an API base URL, a namespace string, a Sentry DSN, a Cloudinary cloud name, a build SHA. VITE_POSTHOG_KEY is deliberately absent from the admin portal, because autocapture on a PII-adjacent surface sends field values to a third party. The rule that follows is mechanical: if a value must not be in a bundle, it must not carry the prefix, and if it carries the prefix it is already published.

The Cloudinary signing secret is where this bites hardest, because signing an upload client-side is the natural design and the provider documents it. It does not happen here. The portal asks the API for a signature, the API

signs with CLOUDINARY_API_SECRET and returns a signature scoped to one folder, one public_id prefix and a short expiry, and the browser uploads with that. A leaked signature lets somebody upload one file to one path rather than rewrite the whole media library.

Rule 8.10.B: the proof that no secret was ever committed is gitleaks over the full history, not over the diff and not an eyeball. A diff scan passes cleanly on a repository whose third commit contains a live key, because the key is not in the diff being scanned; git keeps it forever and every clone carries it. All five repositories are scanned end to end on every push. When gitleaks fires, the response order is rotate first, then rewrite history — never the reverse, because by then the credential is in every clone and removing the commit protects nobody.

8.11 TRANSPORT, HEADERS AND BROWSER POSTURE

TLS, HSTS, the portal CSP, CORS by exact origin, and the one CSRF-exposed route

TLS terminates at Cloudflare and Cloud Run serves nothing over plain HTTP. HSTS is set with a one-year max-age and includeSubDomains on the three portal hosts and on the API host; preload is not requested yet, because a preload entry is effectively permanent and the hostnames are two weeks old. The headers below are asserted by a Playwright check against the deployed hosts on every release, not written once in a config file and trusted, because a header that regresses looks exactly like a header that works.

```
Content-Security-Policy:
  default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';
  img-src 'self' data: https://res.cloudinary.com; font-src 'self' data:;
  connect-src https://api.atavishaala.in https://o0.ingest.sentry.io;
  frame-ancestors 'none'; base-uri 'none'; form-action 'none'; object-src 'none'

Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff      Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(self), camera=(self), microphone=()
```

script-src carries no 'unsafe-inline' and no 'unsafe-eval', which is affordable because Vite emits hashed module files and nothing inlines a script; the CI check fails the build if either appears. style-src keeps 'unsafe-inline' and that is a real weakness stated plainly: the component library sets style attributes, and removing them is a refactor nobody has budgeted. connect-src is an exact allow-list, so an injected script cannot beacon to an attacker origin even if it executes. frame-ancestors none removes clickjacking of an admin action, and it is preferred over X-Frame-Options because the latter has no directive for what we want.

CORS is an exact-origin allow-list read from ALLOWED_ORIGINS, with credentials enabled and no wildcard anywhere — a wildcard is incompatible with credentialed requests in any case, which is a useful accident. An unlisted origin gets no CORS headers at all rather than a 403, because the browser blocking the response is the correct outcome and a 403 body would tell a prober that the endpoint exists. The refresh cookies are host-only, Secure, HttpOnly, SameSite=Strict and named per namespace, so a stolen partner refresh cookie is not sent to admin.atavishaala.in by the browser at all.

8.11.4 The CSRF position: exactly one exposed route

Every mutating endpoint reads its credential from the Authorization header, and a cross-site form post cannot set that header, so the entire API surface except one route is structurally immune to CSRF. The exception is POST /:ns/auth/refresh, which is authenticated by a cookie precisely so that JavaScript cannot read it, and a cookie is exactly what a browser attaches to a cross-site request. Three things protect it. SameSite=Strict means the browser does not attach the cookie to any cross-site request, which alone closes the classic attack. The route requires Content-Type: application/json, which an HTML form cannot produce without a preflight, and the preflight fails the origin allow-list. And the route is not usefully forgeable even if both failed: it returns a new access token in the response body, which the attacker cannot read cross-origin, while rotating the refresh token and so logging the victim out — a nuisance, not an escalation. A CSRF token would add a fourth layer and was rejected as unjustified state for a route whose worst outcome is a forced re-login.

8.12 RATE LIMITING, ABUSE, AND THE UNTRUSTED-FLEET PROBLEM

Per-namespace buckets, and what a hostile registration can reach

Limits are per namespace and per identity rather than per IP, because the interesting abuser holds a valid token and a dispatch office behind one NAT looks like one IP. A 429 carries Retry-After: 30 and the portal kit backs off rather than retrying immediately. The fleet namespace is the tightest of the four for two reasons: it is the only namespace whose users include people the organisation has never met, and it contains the one endpoint that returns plaintext PII, so a limit there is a limit on the worst outcome in the system.

NAMESPACE	READ BUCKET	WRITE BUCKET	TIGHTER SUB-LIMITS AND REASONING
/admin	600 / min	120 / min	Staff are few and trusted; the cap exists to catch a runaway poll loop, not an operator. export.create is 5 jobs per hour per staff user.
/partner	300 / min	60 / min	A warehouse terminal polling a shipment board is the normal load. p.catalog.price is 600 writes per hour per org, because a bulk price update is legitimate.
/fleet	240 / min	60 / min	Tightest namespace. /jobs/:id/manifest is 20 reads per hour per org and 3 per job; offer responses are 120 per hour; auth/otp/request is 5 per mobile per hour.
public, /app	120 / min	30 / min	Unauthenticated surfaces are IP-bucketed at the Cloudflare edge as well, which is the only free layer available and is the reason the origin sees so little of it.

The uncomfortable question is registration. There is no self-registration on the partner namespace at all — an org exists because a staff member invited it — but a delivery company can be created from the public site, which means anyone can obtain a fleet login. That is a deliberate growth decision, so what a hostile registration reaches before KYC approval is enumerated rather than assumed.

WHAT AN UNAPPROVED FLEET ORG CAN DO	REACHABLE	WHY
Sign in, see its own org shell, read GET /fleet/me	yes	The shell renders an onboarding checklist and nothing else. Every board is empty.
Upload its own KYC documents and add its own riders and vehicles	yes	Writes scoped to its own fleet_org_id by the token claim. It is filling in its own form.
Receive job offers, or read GET /fleet/offers	no	Allocation filters on fleet_orgs.status = ACTIVE and kyc_verified_at NOT NULL, in the query and not in the client. An unapproved org is not a candidate in any wave.
Accept a job, or reach any manifest	no	A manifest is job-scoped and it holds no jobs. There is no endpoint that returns a job it was not offered, so there is nothing to guess at.
See any customer, order, price, address or other fleet	no	No f.customer.read, f.order.read or f.payment.read string exists in the union (8.5.12), so no permission can be composed that returns them.
Add a bank account and be paid	partly	It can submit one. Payout release requires an ACTIVE org and a staff-verified account, both gated by step-up, so money cannot leave without a human.

The residual value of a hostile registration is therefore a name in a list and some storage. The real cost is nuisance at scale — a hundred fake orgs make the fleet queue unusable — which is handled by requiring a GSTIN at registration and by the queue defaulting to documents-submitted rather than to everything.

8.13 AUDIT

Invariant I-08: middleware, not discipline. Append-only, masked, and retained

I-08 EVERY ADMIN WRITE IS AUDITED WITH A BEFORE/AFTER DIFF

Any request to /api/v1/admin with a method other than GET writes an audit_log row carrying actor, permission used, target, and a JSON diff. Audit rows are masked by the same extension. Enforcement: Audit is an Express middleware on the admin router, not a per-handler call, so a new endpoint cannot forget it. A test asserts row count grows by one per mutating fixture.

Audit is an Express middleware mounted on the admin router, so every non-GET request under /api/v1/admin writes a row and a new endpoint cannot forget to. That placement is the whole design decision: a per-handler audit call is a convention, and conventions are complied with at roughly the rate people remember them. A test asserts the row count grows by exactly one per mutating fixture, which also catches the double-write a retrieved handler would otherwise hide.

```

audit_log row:
  staff_user_id  permission          method  path                target_type  target_id
1f3a...        partner.status.write  POST    /admin/partners/:id  partner_org  9c2e...
ip_hash (SHA-256 of the IP, never the IP)  user_agent  created_at
diff_json: { "status": { "before": "ACTIVE", "after": "SUSPENDED" },
              "phone": { "before": "****1234", "after": "****1234" } }  <= 8192 bytes

```

Three properties of that row matter. The diff runs through the masking extension before storage, so the register of who changed what is not itself a plaintext PII store — a mistake that is easy to make and impossible to undo once the rows exist. The IP is stored as a hash, which supports "was this the same client" without retaining a location. And the diff is capped at 8192 bytes with truncation recorded, because a bulk update would otherwise write megabytes into a 0.5 GB database (risk R-07).

The table is append-only: the API issues no UPDATE and no DELETE against it, audit.read is the only permission that touches it, and there is deliberately no audit.write string in the union. The one process that removes rows is the monthly audit-archive cron, which exports rows older than 180 days to an R2 bucket as gzipped NDJSON and then deletes them — a retention policy rather than an edit path. pii_disclosures is never archived and never purged; it is small and it is the one register that must outlive an argument.

Deliberately not audited: every GET, because a read log on the admin namespace would be larger than the data and would answer no question the disclosure register does not already answer better; the partner and fleet namespaces, because a counterparty acting inside its own tenant is ordinary business and the money paths are covered by append-only ledgers instead; and login success and failure, which are counters and timestamps on staff_users rather than audit rows. The first of those three is the one that will hurt during an incident: if a staff account is compromised, we can say what it changed and what it unmasked, but not everything it looked at. Gap G-06.

8.14 INCIDENT RESPONSE

What counts, the first five actions in order, and how each actor class is contained

A security incident is any of: a credential believed to be in the wrong hands; a disclosure row that cannot be explained by a job; an export whose size or timing has no owner; a login from a country nobody is in; a gitleaks finding on a live key; or an unexplained write to a money table. A failed login run, a 429 storm and a Cloudflare challenge page are not incidents, they are Tuesday. The distinction is deliberate: a definition that includes noise gets ignored, and an organisation of four cannot afford a process it ignores.

1. Contain the identity before understanding it. Set the staff user to SUSPENDED, or set the fleet org to offer-ineligible. Do not start by reading logs — every minute spent understanding is a minute the session is still live.
2. Revoke every refresh family for that identity. A suspended row stops new logins but an issued refresh token keeps minting access tokens for up to seven days, so suspension without family revoke is not containment. POST /admin/auth/logout-all is the button; the effect is immediate on refresh and at most 900 seconds on the access token in flight.
3. Call the technical lead. One name, no rota, no escalation tree — with four engineers a tree is a delay dressed as a process. The lead owns the decision to rotate keys and the decision to tell anyone outside.
4. Read the audit_log and pii_disclosures for the session window, filtered by actor, and write down the target ids. This is the evidence step and it is third rather than first for the reason in step one. RB-08 is the checklist.
5. Decide on rotation. A leaked JWT_PRIVATE_KEY means every session everywhere, so rotate and accept a global logout. A leaked PII_ENC_KEY has no rotation path (G-01), so the decision is about disclosure rather than remediation, and that is exactly why G-01 is ranked first in 8.15.

Containment differs by actor class. A staff session dies on suspend plus family revoke. A partner org is set to SUSPENDED, which delists its SKUs and blocks its writes while leaving its in-flight shipments visible to ops. A fleet takes two moves rather than one: offer eligibility off, so it enters no further wave, and then its accepted jobs walked off one at a time — because suspension does not take jobs away from a fleet that already holds them, and each of those jobs still carries an open disclosure window. Closing that window is the move that is easy to forget.

DISCLOSURE OBLIGATION — READ THIS AS A PROMPT TO GET ADVICE, NOT AS ADVICE

This is not legal advice and nobody on this team is qualified to give it. What we know: India's Digital Personal Data Protection Act, 2023 places obligations on a data fiduciary to notify the Data Protection Board and affected data principals of a personal data breach, and the CERT-In directions of April 2022 require certain cyber incidents to be reported within six hours of noticing them. Both are materially stricter than the informal handling a pilot invites, and the six-hour figure in particular is short enough that it must be decided in advance rather than looked up during an incident. Action for the programme, not for this document: before go-live, the technical lead obtains a written position from a qualified adviser on which incidents trigger which notification and in what form, and RB-08 gets a step naming it. Until then, assume the obligation exists and that the clock is short.

8.15 THE HONEST SECURITY GAPS

Eight of them, ranked, with the compensating control and what it does not cover

Every item below is a decision, not an oversight, and most of them are decisions bought with the Rs 0 per month constraint of I-14. Ranked by what would hurt most on the day it matters.

ID	GAP	RISK IF IT IS EXERCISED	COMPENSATING CONTROL, AND ITS LIMIT
G-01	No KMS. PII_ENC_KEY is a base64 string in a Cloud Run environment variable, and rotation needs a re-encrypt job that does not exist.	Anyone who can read the service configuration decrypts every governed column. A leaked key cannot be rotated away, so the response is disclosure, not remediation.	Encryption at rest still defeats a leaked backup or a stolen read-replica credential, which is the likelier event. Limit: it does not defeat anyone with GCP project access. Named fix: Cloud KMS, roughly \$0.06 per key-version-month plus operations, needs a waiver.
G-02	Every secret rotation in 8.10 is manual, and two keys have no rotation path at all: PII_ENC_KEY needs a job nobody has written, PII_INDEX_HMAC_KEY is never rotated by design.	A rotation that requires fifteen console visits and a redeploy is a rotation that happens after an incident and not before one. A leaked index key gives an attacker a membership oracle: it confirms whether a given phone number is in the database.	Nothing is shared between environments, the JWT keypair can be rotated with a JWKS overlap, and the index key is useless without a candidate value in hand. Limit: there is no re-encrypt and no re-index job, so neither can be fixed under pressure — both must be built first.
G-03	No WAF rules beyond the Cloudflare free managed set.	Application-layer attacks that look like valid traffic are unfiltered at the edge; the origin is the only defence.	The API is JSON-only, Zod-validated on every route, and parameterised through Prisma, so the classes a WAF catches are mostly structurally absent. Limit: no virtual patching, so a dependency CVE must be fixed in code and deployed.
G-04	No penetration test, internal or external.	Unknown unknowns. Nothing in the set has been attacked by someone trying to break it rather than build it.	Twelve CI gates, the contract sweep asserting 401 for three wrong audiences, and the week-nine PII sweep P9-BE1-01. Limit: all three test what we thought of. A basic external test is roughly Rs 60 000 and is the highest-value paid item here.
G-05	No SIEM. The disclosure anomaly check is a cron and a human reading a report.	A slow exfiltration inside the manifest window is invisible until somebody looks, and nobody looks on a Sunday.	The register is complete and immutable, so detection is late rather than impossible and the evidence survives. Limit: latency is measured in days, and the alert threshold is a trailing average with no tuning behind it.
G-06	GET requests on the admin namespace are not audited (8.13).	After a staff compromise we can prove what was changed and what was unmasked, but not everything that was read.	Everything readable is masked, so the value of a read is low, and exports are capped at 5 000 rows and are audited. Limit: an operator can still page through a list surface indefinitely.
G-07	A legitimate operator can enumerate masked records at volume, and nothing in this design stops them.	An AUDITOR or OPS_AGENT seat can walk every customer, order and shipment page by page. Masked, but a complete relationship graph with last-four identifiers.	Nothing preventive. Rate limits slow it, exports cap it, and audit records the mutations it never makes. This is the gap the charter cannot close, and it is stated rather than mitigated.
G-08	TOTP is the only second factor. There is no WebAuthn or hardware-key option for staff, not even for the two SUPER_ADMIN holders.	TOTP is phishable in real time: a proxy page collects the password and the six digits and replays both inside the 30-second window. That is R-10, and it is the most plausible route to a compromised admin session.	Mandatory TOTP still defeats credential stuffing and password reuse, which is the common case; last_used_step blocks replay of an observed code; and step-up limits what one stolen session reaches without a second prompt. Limit: none of that stops a live proxy. WebAuthn costs nothing in licence fees and roughly three dev-days, and is the highest-value unpaid item on this list.

Read the list as a ranking rather than a backlog. G-01 and G-08 are the two that change the outcome of a real incident, and both have named fixes with prices attached: a KMS key at a few cents a month, and three dev-days for WebAuthn. Everything else on the list degrades detection or response speed rather than the boundary itself.

What this charter does buy, and it is worth stating at the end of a page of gaps: a stolen session of any kind reaches masked data only, because the mask is at the ORM boundary and no permission exists to lift it. That is the one property the whole document exists to protect, and none of the eight gaps above defeats it. G-01 defeats it only for someone who already has the production configuration, at which point the mask was never the control. Review this section at the Friday gate in week nine, when P9-BE1-01 and P9-BE1-02 have run and the list can be checked against what the tests actually found.